

Data Structures

Linked Lists

- Review on Pointer, Dynamic Memory allocation and De-allocation
- Singly Linked Lists
- Doubly Linked Lists

- **Structures**
- Structures are aggregate data types built using elements of primitive data types.
- Structure are defined using the **struct** keyword:

E.g.

```
struct Time{  
    int hour;  
    int minute;  
    int second;  
};
```

- The **struct** keyword creates a new user defined data type that is used to declare variables of an aggregate data type.
- Structure variables are declared like variables of other types.
- **Syntax:**

struct <structure tag> <variable name>;

- E.g. **struct Time timeObject,**
- **struct Time *timeptr;**

Accessing Members of Structure Variables

- **The Dot operator (.):**

- to access data members of **structure variables**.

The Arrow operator (->):

- to access data members of **pointer variables** pointing to the structure.

E.g. Print member hour of timeObject and timeptr.

```
cout<< timeObject.hour;
```

or

```
cout<<timeptr->hour;
```

TIP: `timeptr->hour` is the same as `(*timeptr).hour`.

The parentheses is required since `(*)` has lower precedence than `(.)`.

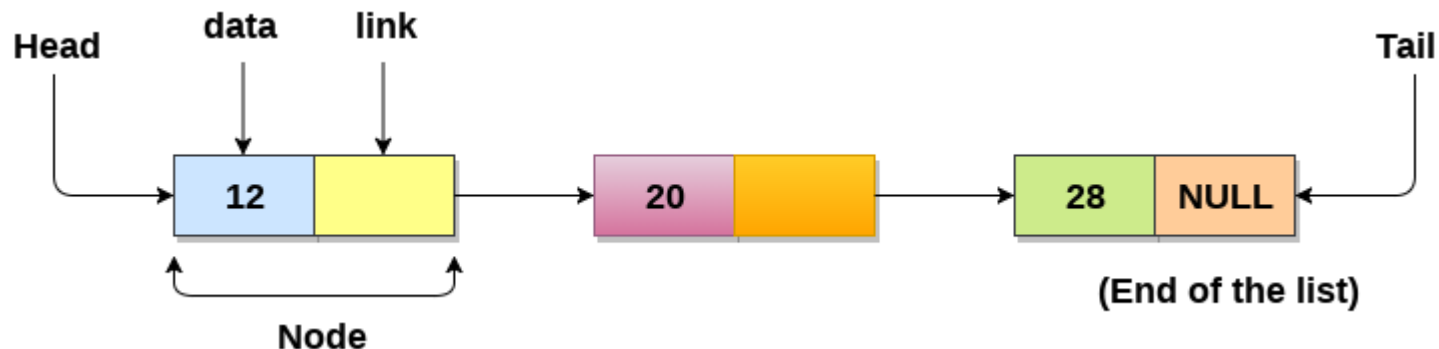
Self-Referential Structures

- Structures can hold pointers to instances of themselves.

```
struct node{  
    char name[10];  
    int count;  
    struct node *next;  
};
```

Singly Linked Lists

- Linked lists are the most basic self-referential structures.
- Linked lists allow you to have a chain of structs with related data
- A linked list is a **linear data structure**, in which the elements **are not stored at contiguous memory locations**.
- The elements in a linked list are linked using pointers as shown in the below image:



- In simple words, a linked list consists of nodes where each node contains a data field and a reference(link) to the next node in the list.

Why Linked List?

- Arrays can be used to store linear data of similar types, but arrays have the following limitations:
 - **The size of the arrays is fixed**: So we must know the upper limit on the number of elements in advance. Also, generally, the allocated memory is equal to the upper limit irrespective of the usage.
 - **Insertion of a new element / Deletion of a existing element in an array of elements is expensive**: The room has to be created for the new elements and to create room existing elements have to be shifted but in Linked list if we have the head node then we can traverse to any node through it and insert new node at the required position.

Example:

- In a system, if we maintain a sorted list of IDs in an array `id[] = [1000, 1010, 1050, 2000, 2040]`.
- If we want to insert a new ID 1005, then to maintain the sorted order, we have to move all the elements after 1000 (excluding 1000).
- Deletion is also expensive with arrays until unless some special techniques are used. For example, to delete 1010 in `id[]`, everything after 1010 has to be moved due to this so much work is being done which affects the efficiency of the code.

- **Advantages of Linked Lists over arrays:**
 - Dynamic Array.
 - Ease of Insertion/Deletion.

Singly Linked Lists

- Linked lists are the most basic self-referential structures.
- Linked lists allow you to have a chain of structs with related data

Array vs. Linked lists

- Arrays are simple and fast but we must specify their size at construction time.
- This has its own drawbacks.
- If you construct an array with space for n , tomorrow you may need $n+1$. Here comes a need for a more flexible system?

Drawbacks of Linked Lists:

- **Random access is not allowed.** We have to access elements sequentially starting from the first node(head node). So **we cannot do a binary search** with linked lists efficiently with its default implementation.
- **Extra memory** space for a pointer is required with each element of the list.
- It takes a lot of time in traversing and changing the pointers. Reverse traversing is not possible in singly linked lists.
- **Direct access to an element is not possible** in a linked list as in an array by index.
- Searching an element is costly and requires $O(n)$ time complexity. **Sorting** of linked list is very complex and costly.

Types of Linked Lists:

Simple Linked List – In this type of linked list, one can move or traverse the linked list in only one direction. It is also called “*Singly Linked List*”.

Doubly Linked List – In this type of linked list, one can move or traverse the linked list in both directions (Forward and Backward)

Circular Linked List – In this type of linked list, the last node of the linked list contains the link of the first/head node of the linked list in its next pointer and the first/head node contains the link of the last node of the linked list in its prev pointer

- Basic operations on Linked Lists:

- Deletion
- Insertion
- Search
- Display

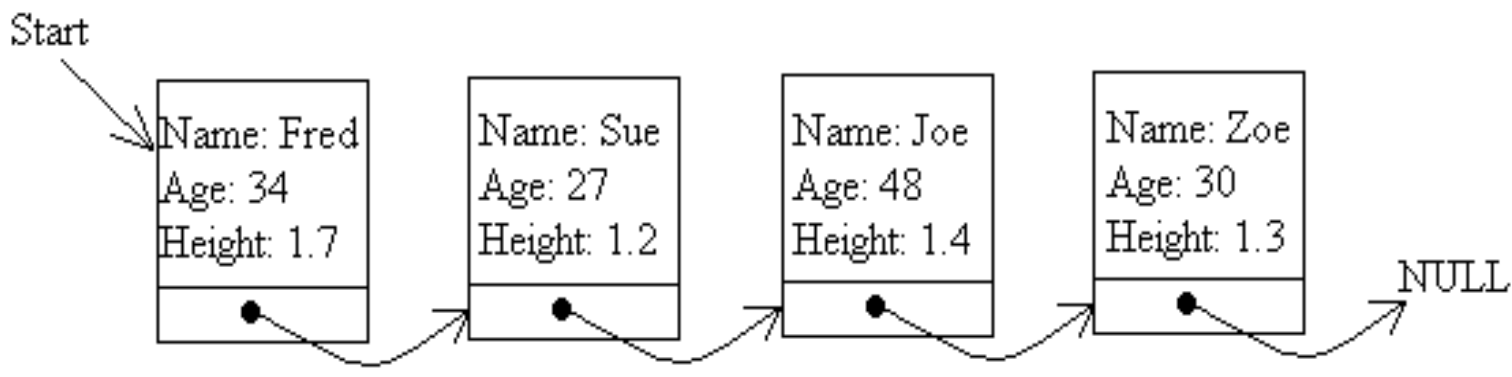
Singly Linked Lists

Representation of Singly Linked Lists:

- linked list is represented by a pointer to the first node of the linked list. **The first node** is called **the head of** the linked list. If **the linked list is empty**, then the value of the head points **to NULL**.
- **Each node in a list consists of at least two parts:**
 - A Data Item (we can store integer, strings, or any type of data).
 - Pointer (Or Reference) to the next node (connects one node to another) or An address of another node

Creating Linked Lists in C++

- A linked list is a data structure that is built from structures and pointers.
- It forms a chain of "nodes" with pointers representing the links of the chain and holding the entire thing together.
- A linked list can be represented by a diagram like this one:



Defining the data structure for a linked list

```
struct node
{ char name[20];    // Name of up to 20 letters
  int age
  float height;    // In metres
  node *next; // Pointer to next node
};
struct node *head =NULL
```

Adding a node to the list

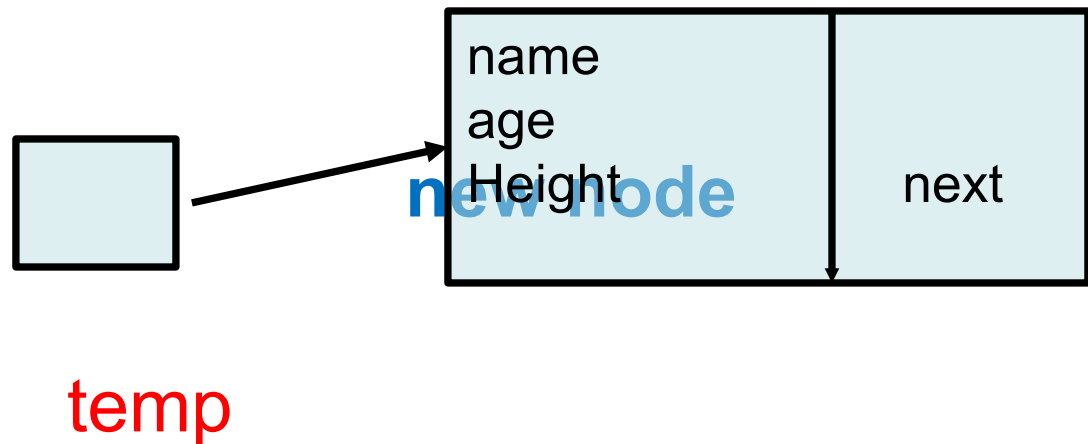
- i. insertAtBeginning
- ii. insertAtEnd
- iii. insertAtferPosition
- iv. insertBeforePosition

insertAtBeginning

Steps:

1. Declare the space for a pointer item and assign a temporary pointer to it.
- This is done using the new statement as follows:

```
node *temp = new node
```

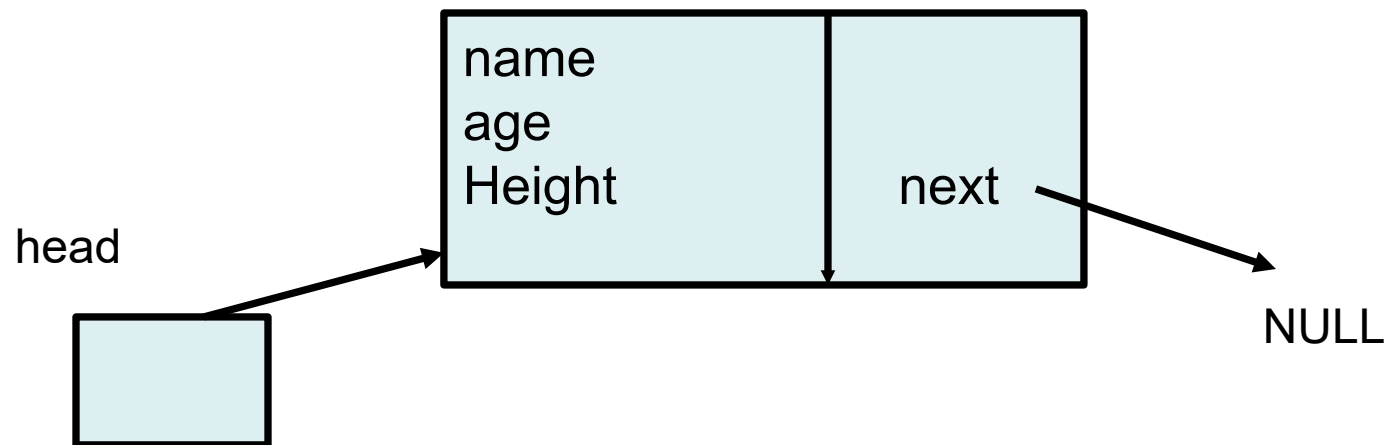


2. Ask the user to fill in the details of the person, i.e. the name, age, address or whatever:

```
cout << "Please enter the name of the person: ";  
cin >> temp->name;  
cout << "Please enter the age of the person : ";  
cin >> temp->age;  
cout << "Please enter the height of the person : ";  
cin >> temp->height;  
temp->next = NULL;
```

3. Set the pointer. For this we need to consider different cases.
Case 1: When the new node is added to an empty list.
i.e

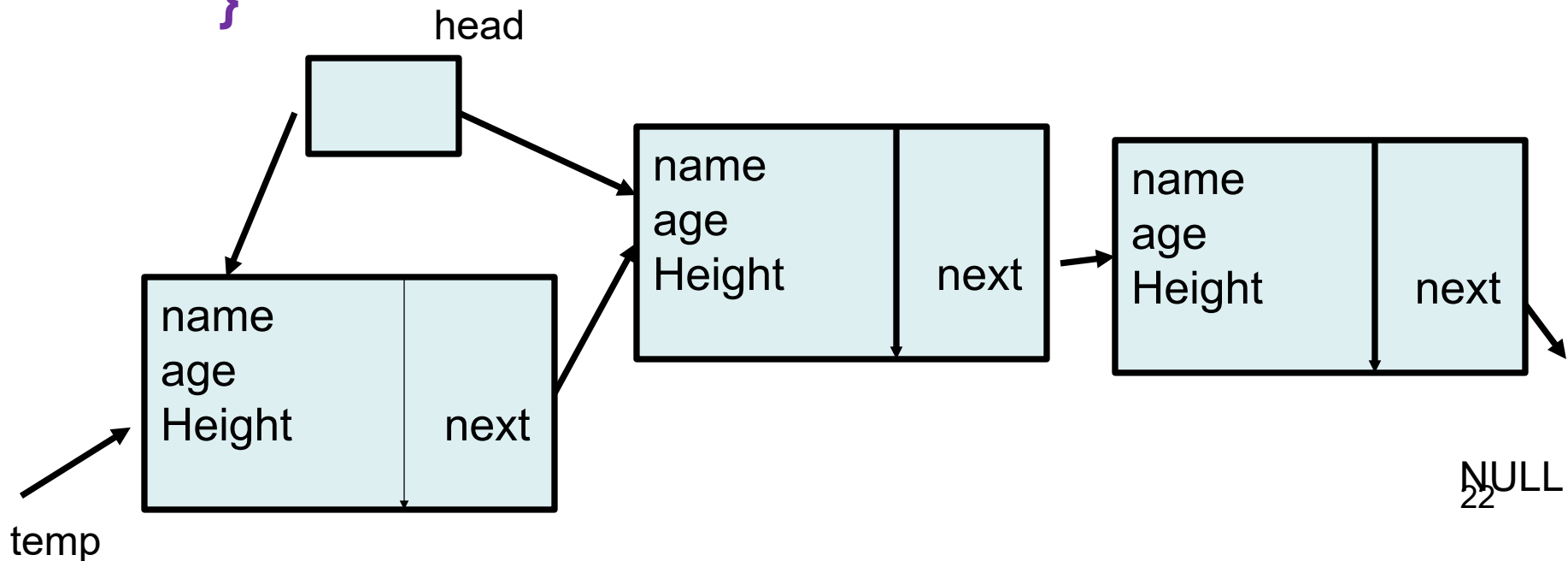
```
if(head==NULL){  
    temp->next=NULL;  
    head=temp;  
}
```



Case 2: Adding a new node at the beginning of a list that contains at least one node.

i.e

```
if(head!=NULL){  
    temp->next=head;  
    head=temp;  
}
```



//C++ Implementation of inserting a new node at the beginning of a //singly //linked list. Use the structure given below.

```
int totalNodes=0;
struct node{
    int id;
    node* next;
}*head=NULL;
```

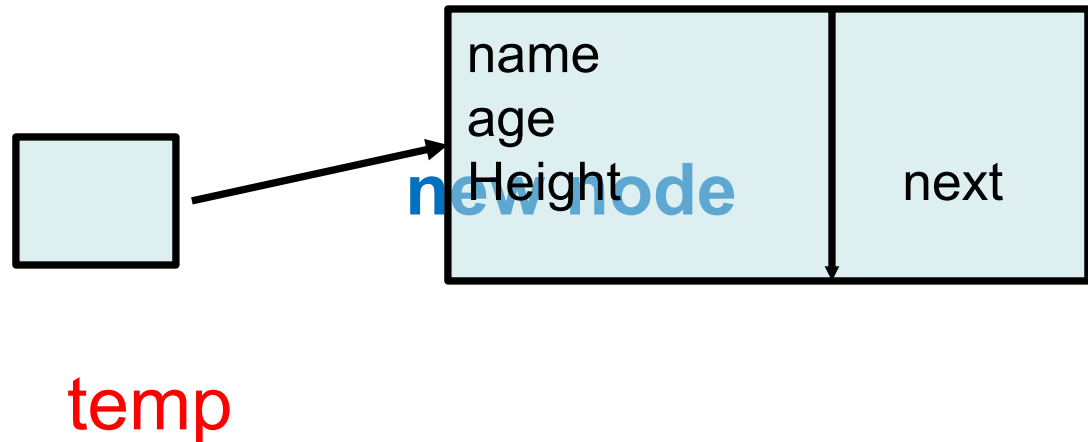
```
void insertBeginning(){
    node* temp=new node;
    cout<<"Enter Id:";
    cin>>temp->id;
    if(head==NULL){
        temp->next=NULL;
    }
    else{
        temp->next=head;
    }
    head=temp;
    totalNodes++;
}
```

insertAtEnd

Steps:

1. Declare the space for a pointer item and assign a temporary pointer to it.
 - This is done using the new statement as follows:

```
node *temp = new node
```

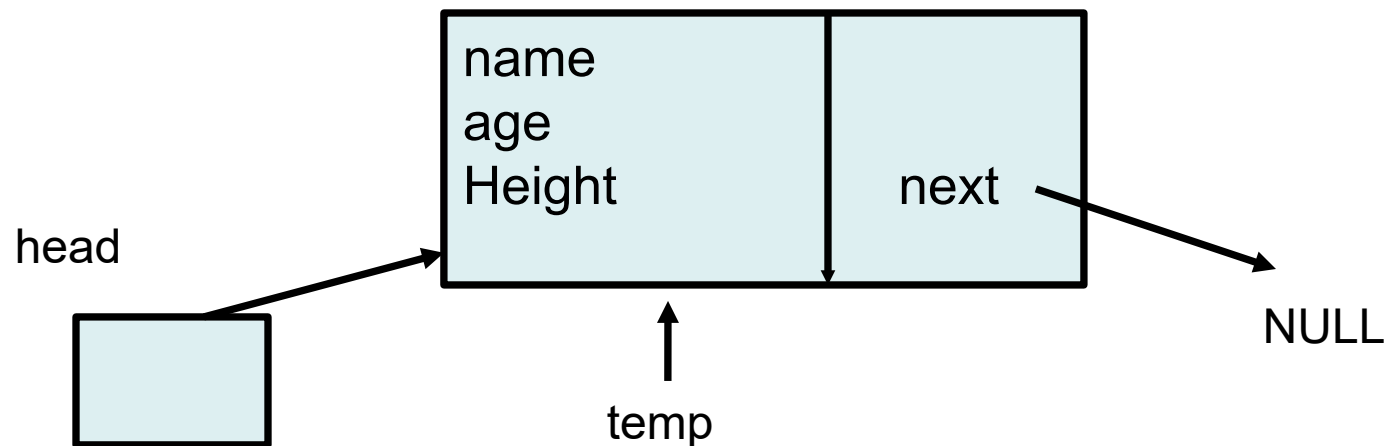


2. Ask the user to fill in the details of the person, i.e. the name, age, address or whatever:

```
cout << "Please enter the name of the person: ";  
cin >> temp->name;  
cout << "Please enter the age of the person : ";  
cin >> temp->age;  
cout << "Please enter the height of the person : ";  
cin >> temp->height;  
temp->next = NULL;
```

3. Set the pointer. For this we need to consider different cases.
Case 1: When the new node is added to an empty list.
i.e

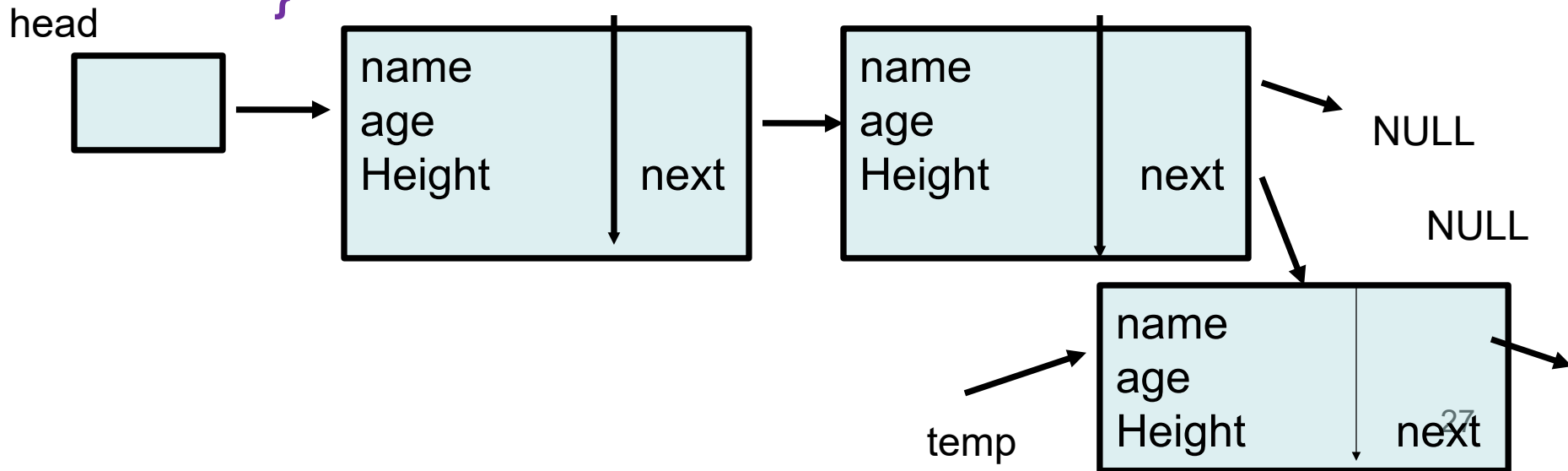
```
if(head==NULL){  
    temp->next=NULL;  
    head=temp;  
}
```



Case 2: Adding a new node at the end of a list that contains at least one node.

i.e

```
if(head!=NULL){  
    node* curr=head;  
    temp->next=NULL;  
    while(curr->next!=NULL){  
        curr=curr->next;  
    }  
    curr->next=temp;  
}
```



//C++ Implementation of inserting a new node at the end of a
//singly linked list. Use the structure given below.

```
int totalNodes=0;
struct node{
    int id;
    node* next;
}*head=NULL;
```

```
void insertEnd(){
    node* temp=new node;
    cout<<"Enter Id:";
    cin>>temp->id;
    temp->next=NULL;
    if(head==NULL){
        head=temp;
    }
    else{
        node* curr=head;
        while(curr->next!=NULL){
            curr=curr->next;
        }
        curr->next=temp;
    }

    totalNodes++;
}
```

Inserting a node before a given node

```
void insert_beforeNode(){
    int location;
    cout<<"Insertion before which node?";
    cin>>location;
    node* temp=getNode();
    if(head==NULL){
        temp->next=NULL;
        head=temp;
    }
```

```
else if(location==1){
    temp->next=head;
    head=temp;
}
else{
    node* curr=head;
    for(int i=1;i<(location-1);i++){
        curr=curr->next;
    }
    temp->next=curr->next;
    curr->next=temp;
}
}
```

```
node* getNode(){  
    node* temp=new node;  
    cout<<"Name:";  
    cin>>temp->name;  
    cout<<"Id :";  
    cin>>temp->id;  
    return temp;  
  
}
```

Inserting a node after a given node

Exercise

Deleting a node from the list

- we have three choices:
 - Delete a node from the start of the list,
 - delete one from the end of the list, or
 - delete one from somewhere in the middle.
- When a node is deleted, the space that it took up should be reclaimed. Otherwise the computer will eventually run out of memory space. This is done with the delete instruction:

delete temp; // Release the memory pointed to by temp

Deletion in singly linked list at beginning

- Deleting a node from the beginning of the list is the simplest operation of all.
- Since the first node of the list is to be deleted, therefore, we just need to make the head, point to the next of the head. This will be done by using the following statements.

temp= head;

head = temp->next;

delete temp;

Algorithm

Step 1: IF head = NULL

Write UNDERFLOW

Go to Step 5

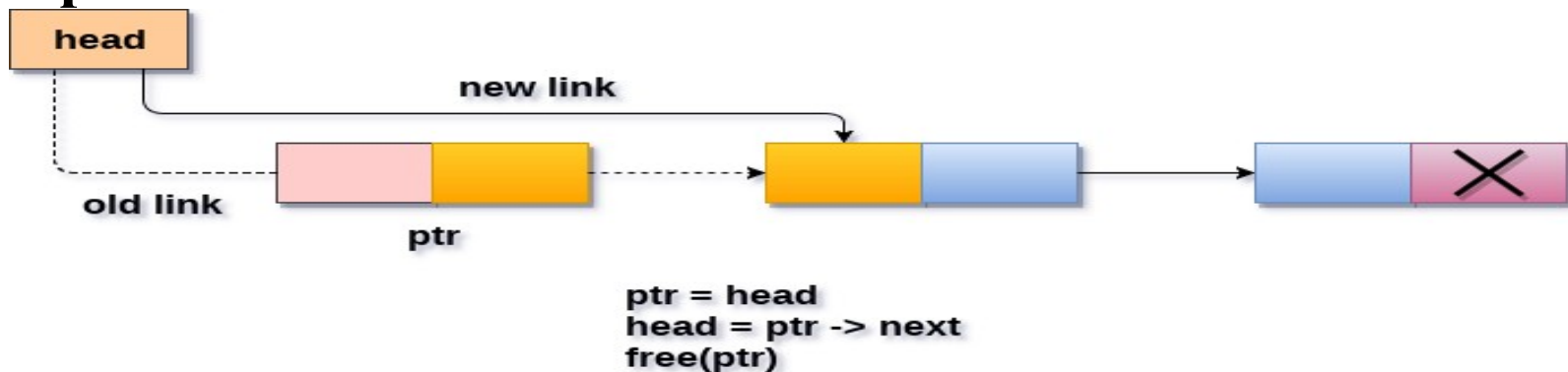
[END OF IF]

Step 2: SET temp = head

Step 3: SET head = head->next

Step 4: delete temp

Step 5: EXIT



Deleting a node from the beginning

```
void delete_start_node()
{
    node *temp;
    temp = head;
    head = head->next
    delete temp;
}
```

Deletion in singly linked list at the end

- There are two scenarios in which, a node is deleted from the end of the linked list.
 - There is only one node in the list and that needs to be deleted.
 - There are more than one node in the list and the last node of the list will be deleted.

- In the first scenario,

- the condition

head $\rightarrow$ **next** = **NULL** will survive and therefore, the only node head of the list will be assigned to null. This will be done by using the following statements.

temp = **head** ;

head = **NULL** ;

delete temp;

- In the second scenario,
- The condition **head** → **next** = **NULL** would fail and therefore, we have to traverse the node in order to reach the last node of the list.
- For this purpose, just declare a temporary pointer temp and assign it to head of the list.
- We also need to keep track of the **second last node** of the list. For this purpose, two pointers temp and temp1 will be used where temp will point to the last node and temp1 will point to the second last node of the list.
- this all will be done by using the following statements.

```

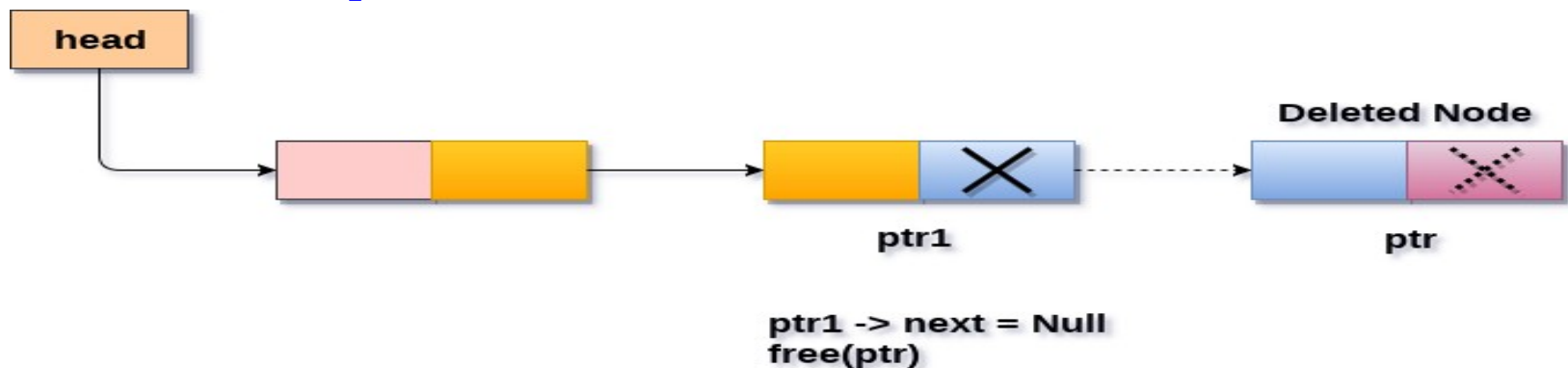
temp= head;
while(temp->next != NULL)
{
    temp1 = temp;
    temp = temp->next;
}

```

- Now, we just need to make the pointer temp1 point to the NULL and the last node of the list that is pointed by temp will become free. It will be done by using the following statements.

```
temp1->next=null;
```

```
delete temp;
```



Deleting a node from the last

Algorithm

Step 1: IF HEAD = NULL

 Write UNDERFLOW

 Go to Step 8

[END OF IF]

Step 2: SET temp = head

Step 3: Repeat Steps 4 and 5 while temp -> next != NULL

Step 4: SET temp1 = temp

Step 5: SET temp = temp->next

[END OF LOOP]

Step 6: SET temp1 -> NEXT = NULL

Step 7: delete temp

Step 8: EXIT

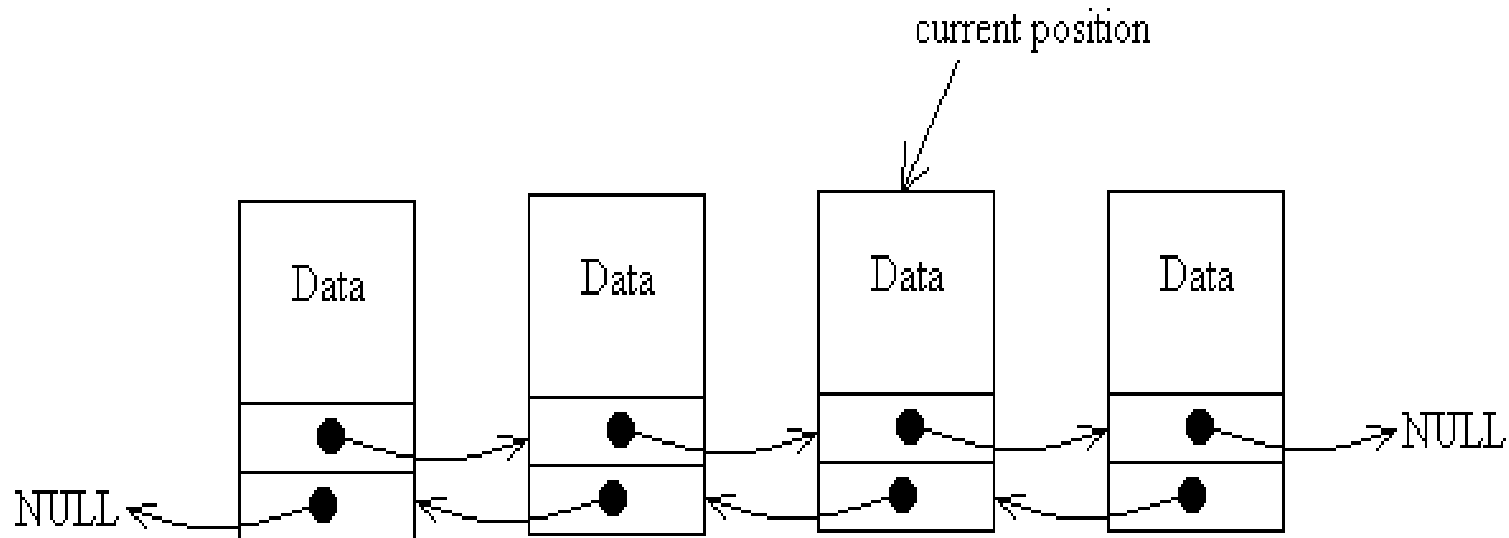
```

void delete_end_node()
{ node *temp1, *temp2;
  if (start_ptr == NULL)
    cout << "The list is empty!" << endl;
  else
    { temp1 = start_ptr;
      if (temp1->nxt == NULL)    // This part is new!
        { delete temp1;
          start_ptr = NULL;
        }
      else
        { while (temp1->nxt != NULL)
            { temp2 = temp1;
              temp1 = temp1->nxt;
            }
          delete temp1;
          temp2->nxt = NULL;
        }
    }
}

```

Doubly Linked Lists

- A doubly linked list is one where there are links from each node in both directions



. Creating Doubly Linked Lists

The nodes for a doubly linked list would be defined as follows:

```
struct node{  
    char name[20];  
    node *nxt; // Pointer to next node  
    node *prv; // Pointer to previous node  
};
```

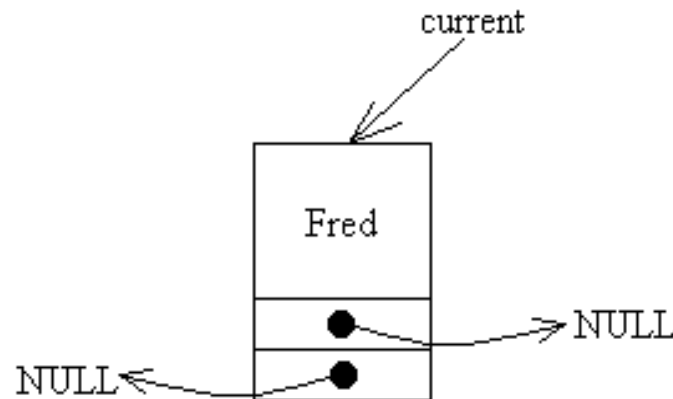
```
node *current;
```

```
current = new node;
```

```
current->name = "Fred";
```

```
current->nxt = NULL;
```

```
current->prv = NULL;
```



Adding a Node to a Doubly Linked List

void add_node_at_start (string new_name)

```
{ // Declare a temporary pointer and move it to the start
  node *temp = current;
  while (temp->prv != NULL)
    temp = temp->prv;
  // Declare a new node and link it in
  node *temp2;
  temp2 = new node;
  temp2->name = new_name; // Store the new name in the node
  temp2->prv = NULL;      // This is the new start of the list
  temp2->nxt = temp;      // Links to current list
  temp->prv = temp2;
}
```

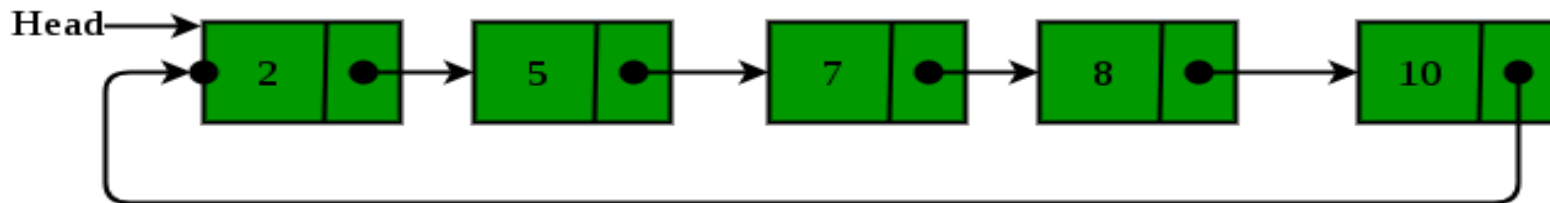
```

void add_node_at_end ()
{ // Declare a temporary pointer and move it to the end
  node *temp = current;
  while (temp->nxt != NULL)
    temp = temp->nxt;
  // Declare a new node and link it in
  node *temp2;
  temp2 = new node;
  temp2->name = new_name; // Store the new name in the node
  temp2->nxt = NULL;      // This is the new start of the list
  temp2->prv = temp;      // Links to current list
  temp->nxt = temp2;
}

```

Circular Linked List

- The **circular linked list** is a linked list where all nodes are connected to form a circle.
- In a circular linked list, the first node and the last node are connected to each other which forms a circle. There is no NULL at the end.



- **Operations on the circular linked list:**
- We can do some operations on the circular linked list similar to the singly linked list which are:
 - Insertion
 - Insertion at the beginning of the list
 - Insertion at the end of the list
 - Insertion in between the nodes
 - Deletion
 - Delete the node only if it is the only node in the circular linked list:
 - Deletion of the last node

Insertion in a circular linked list:

A node can be added in three ways:

Insertion in an empty list

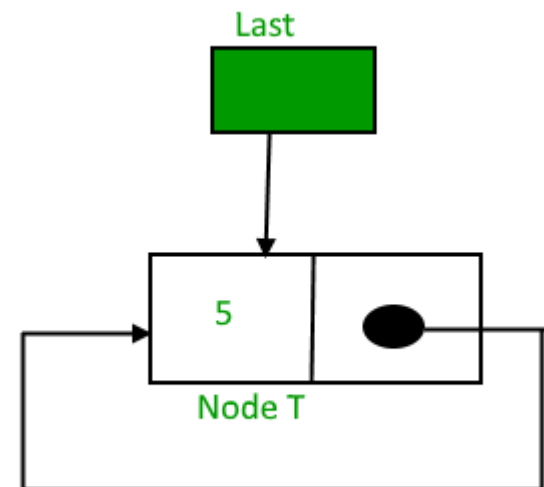
Insertion at the beginning of the list

Insertion at the end of the list

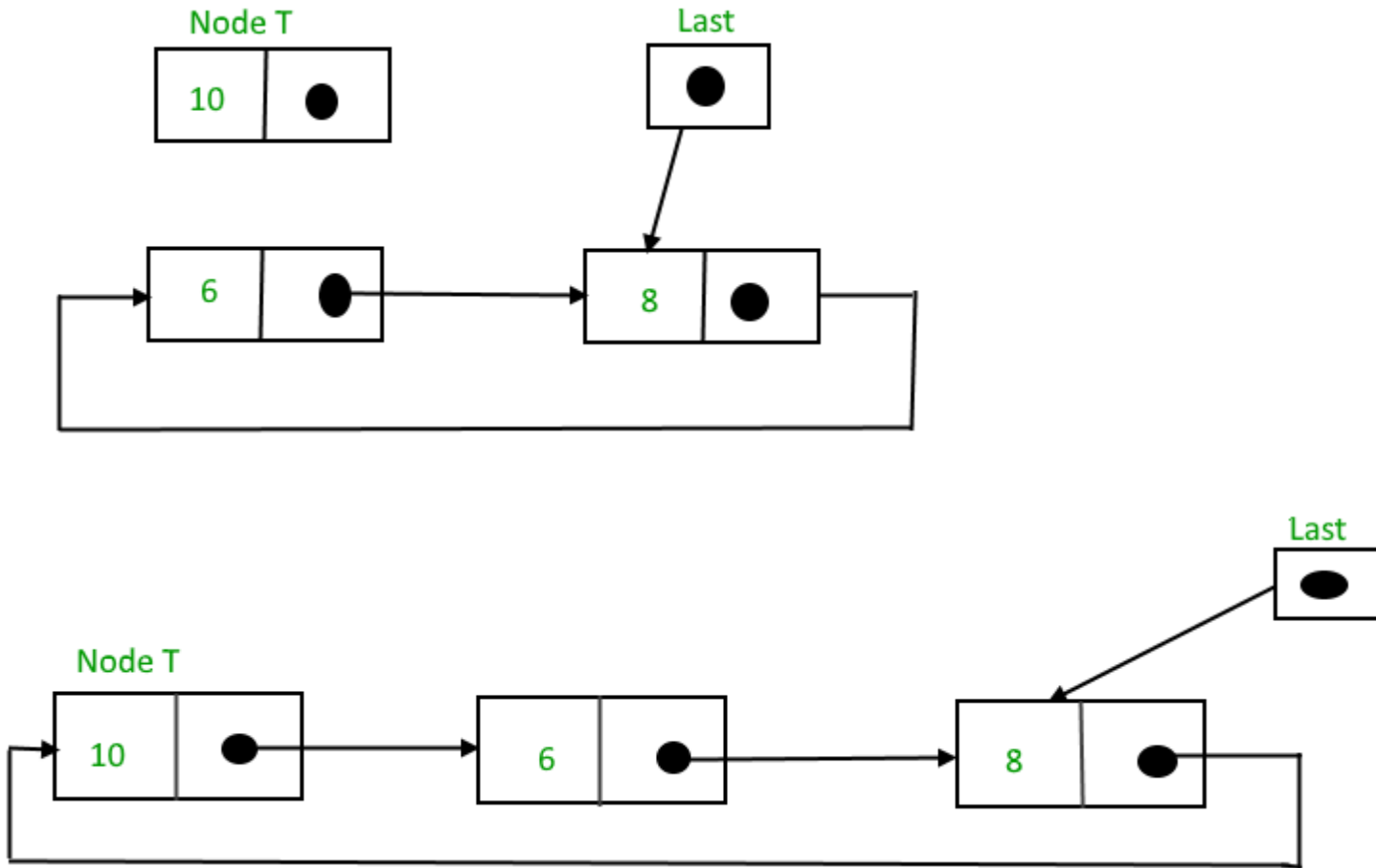
Insertion in between the nodes

Insertion in an empty List:

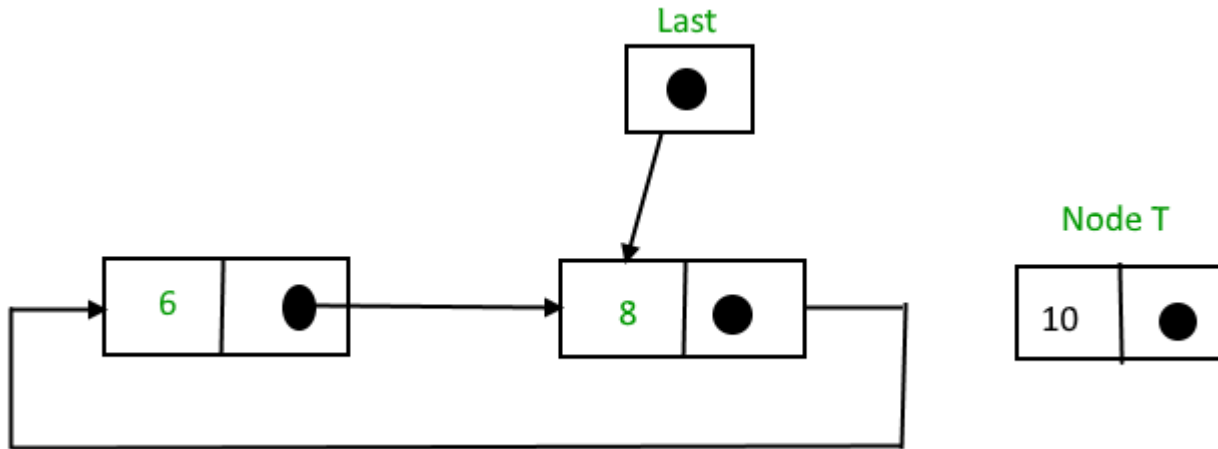
Initially, when the list is empty, the last pointer will be NULL.



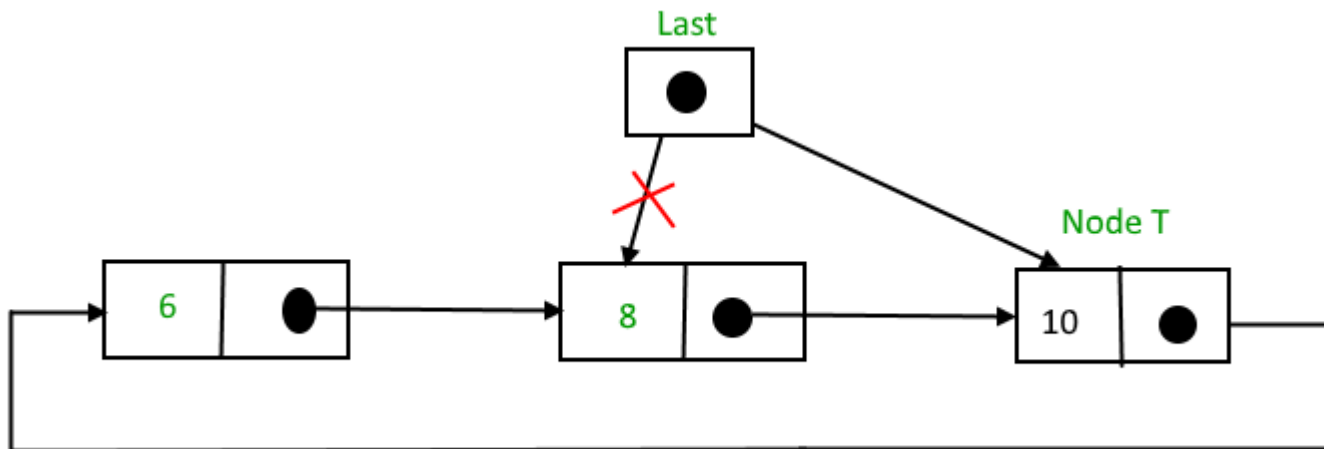
Insertion at the beginning of the list



Insertion at the end of the list



After insertion



2. Deletion of the last node:

- Locate the node before the last node (let it be temp)
- Keep the address of the node next to the last node in temp
- Delete the last memory
- Put temp at the end

3. Delete any node from the circular linked list:

Algorithm:

Case 1: List is empty.

- If the list is empty we will simply return.

Case 2: List is not empty

- If the list is not empty then we define two pointers **curr** and **prev** and initialize the pointer **curr** with the **head** node

- Traverse the list using curr to find the node to be deleted and before moving to curr to the next node, every time set prev = curr.
- If the node is found, check if it is the only node in the list. If yes, set head = NULL and free(curr).
- If the list has more than one node, check if it is the first node of the list. Condition to check this(curr == head).
- If yes, then move prev until it reaches the last node. After prev reaches the last node, set head = head -> next and prev -> next = head. Delete curr.
- If curr is not the first node, we check if it is the last node in the list. Condition to check this is (curr -> next == head).

- If curr is the last node. Set prev -> next = head and delete the node curr by free(curr).
- If the node to be deleted is neither the first node nor the last node, then set prev -> next = curr -> next and delete curr.